Processor Design Exam Playbook

Which architectural feature to choose, for which scenario, and why

This is an open-book *decision manual*. The exam will hand you a scenario (workload, power/cost/area limits, compatibility needs, latency numbers) and ask you to configure a processor and justify the choices. Every part therefore follows the same pattern: (1) a **decision map** (constraint → feature), (2) each feature with **what it fixes / what it costs / when it wins / when it loses**, (3) the **numbers from the slides** you can quote, (4) **exam traps**. Section 7 puts it all together with full worked scenarios and a reusable answer template.

Conventions. *Slides:* tags at the end of a section tell you which deck to open (deck names are listed in Appendix B). A red dagger† marks something that is *not* stated in the slides (my general knowledge or my own arithmetic on slide numbers); it is correct as far as I know, but treat it as a supplement and prefer slide wording in your answer.

On "guaranteed". No document can guarantee a result. What this one does is make every recommendation traceable to the professor's own material, and give you a method (Section 1) that forces you to justify with constraints, numbers and trade-offs, which is what a design question rewards.

1 How to Attack a Design Question

1.1 The six-step answer template

Every "design a processor for this scenario" question can be answered with the same skeleton. Examiners award marks for *linking a constraint to a feature with a reason*, so do this explicitly and in this order.

1. **Classify the machine.** Which class from the course: personal mobile device (PMD), desktop, server, cluster/warehouse-scale, supercomputer, embedded? The class fixes what to optimise: PMD = energy and real-time; desktop = price-performance; server = availability, scalability, throughput; embedded = price.
2. **Classify the workload's parallelism.** Lots of independent requests/threads (TLP/RLP)? Regular loops over arrays (DLP, loop-level ILP)? A single branchy thread (ILP-limited)? Memory-bound with poor locality? This decides core style (wide OoO vs. many small cores vs. multithreaded).
3. **Find the bottleneck with a number.** Use the processor-performance equation and AMAT: is CPI dominated by data hazards, branch penalty, or memory stalls? Quote $f_{branch} \times penalty$, AMAT, miss rate, etc.
4. **Choose one feature per bottleneck** (from the decision maps in the sections below), and state *why it matches this workload*.
5. **State the cost and risk** of each choice (area, power, clock-rate impact, complexity, compatibility). A design with no stated cost looks naive.
6. **Name what you rejected and why** ("VLIW rejected because binary compatibility is required"). This is where a good answer separates from a good-enough answer.

One-sentence justification pattern. "Because the workload is X (evidence: number/constraint), the bottleneck is Y; therefore I choose feature, which reduces CPI term / AMAT term / power by mechanism, at the cost of cost; I rejected alternative because constraint."

1.2 Master lookup: constraint keyword → what it implies

Scan the scenario for these keywords first. The right-hand column is the default direction; later chapters refine it.

If the scenario says...	Lean towards...	Lean away from...
Battery / energy / thermal limit / PMD	Simple or modestly-wide core, small caches, few cache levels, way-predicted caches, DVFS, power gating, turning off idle cores, multicore of small cores instead of one huge core	Very wide issue, deep speculation, trace cache, large windows, aggressive prefetch that burns bandwidth (ILP techniques are *energy-inefficient*: slide "Limits to ILP")
Low cost / embedded / price-sensitive	Simple pipeline, static branch prediction / delay slot, direct-mapped small cache, VLIW if code is fixed and compiler is good (simple hardware)	Tomasulo+ROB, SMT, large L2/L3, large BTB
Server, throughput, many independent requests, database, web	Multicore + multithreading (fine-grained or SMT), large shared L2/L3, interleaved/banked memory, non-blocking caches, dependability (availability)	Spending area on single-thread ILP (diminishing returns)
Single-thread latency / desktop / gaming / one big program	Wide dynamically-scheduled speculative superscalar (Tomasulo+ROB), good branch predictor (two-level / gshare + BTB + return-address stack), small fast L1 + big L2, non-blocking caches, prefetch	Coarse-grained multithreading (does nothing for one thread), VLIW (if code is irregular)

If the scenario says...	Lean towards...	Lean away from...
Regular FP loops, arrays, scientific, matrix	Loop unrolling + software pipelining + compiler scheduling, pipelined FP units, blocking/interchange/fusion, hardware or compiler prefetch, interleaved memory, VLIW is viable; (vector/SIMD/GPU†)	Large dependence on branch prediction (loops are predictable anyway), big victim caches (conflicts are fixable in software)
Binary compatibility across generations / legacy software	Dynamically scheduled superscalar (hardware finds parallelism; same code runs on next generation)	VLIW (breaks when number of units or latencies change)
Unpredictable latencies (cache misses) / data-dependent latencies	Dynamic scheduling (scoreboard/Tomasulo), non-blocking cache, multithreading to hide stalls	Static scheduling relying on fixed latencies; VLIW lock-step (one stall stalls everything)
Deep pipeline / very high clock	Strong branch prediction (misprediction cost grows with depth), BTB, small simple L1 so hit time fits cycle, pipelined cache access	Delayed branch (slots become hard to fill), large associative L1 (hit time ↑)
High branch frequency (compilers, interpreters, control code)	Dynamic predictor: 2-bit counters → correlating / two-level / gshare; BTB; return-address predictor	Predict-not-taken alone, 1-bit predictor
Many outstanding misses possible; out-of-order core present	Non-blocking cache (hit-under-miss / miss-under-miss), multiple memory banks	Blocking cache (wastes the OoO core)
Large working set, poor locality, pointer-chasing	Multithreading to hide latency (fine-grained/SMT), large last-level cache, victim cache, deeper hierarchy	Blocking/tiling and stream prefetch (need regular access patterns)
Memory bandwidth limited, sequential accesses	Interleaved or wide main memory, larger blocks (up to the AMAT optimum), prefetch stream buffers	Very large blocks in a small cache
Limited die area	Multicore of smaller cores (each core simpler, easier to design/manufacture), small-area predictors, SMT as cheap TLP (Power5: SMT cost ≈24% core area)	Wide issue (logic and ports grow faster than issue width)
Real-time / predictability	Static techniques (fixed delay slots, static prediction, static schedule); small caches with known behaviour†	Heavy speculation and large caches with variable timing†
High availability (24/7)	Redundancy, ECC memory, fast repair; compute availability = MTTF/(MTTF+MTTR)	—

Common failure mode: Choosing "the most advanced thing" for every box. The exam rewards *fit*. A PMD answer containing a 12-wide speculative core and a trace cache loses marks; so does a server answer built around a single fast core with no multithreading.

2 Fundamentals, Constraints and Trade-offs

This chapter holds the *constraints* an exam scenario will give you (power, cost, availability, class of machine) and the quantitative tools to reason about them. Everything later is a way of improving one term of the equations here.

2.1 Classes of computers: what each one optimises

Class	Emphasis (slides)	Design consequence
Personal mobile device (phones, tablets)	Energy efficiency, real-time	Low-power cores, small/few cache levels, DVFS and power gating; throughput per joule beats peak speed.
Desktop	Price-performance	Single-thread performance still matters; balanced OoO core with moderate multicore.
Server	Availability, scalability, throughput	Multicore/multithreading, large caches, high memory bandwidth, ECC/redundancy.
Cluster / warehouse-scale (SaaS)	Availability, price-performance	Request-level parallelism; cost and energy per request dominate.
Supercomputer (sub-class)	FP performance, fast internal network	Wide FP units, high memory bandwidth, data-parallel structure.
Embedded	Price	Cheapest design that just meets the requirement; simple pipeline, small caches.

Slides: ca01 "Classes of Computers"

2.2 Kinds of parallelism and which hardware exploits them

- **Application-level:** data-level parallelism (DLP: same operation on lots of data) and task-level parallelism (TLP: independent tasks).
- **Architecture-level:** instruction-level parallelism (ILP: pipelining, superscalar, VLIW), vector/GPU (DLP), thread-level parallelism (multithreading, multicore), request-level parallelism (clusters/WSC).

- **Flynn's taxonomy:** SISD; SIMD (vector architectures, multimedia extensions, GPUs); MISD (no commercial implementation); MIMD (tightly-coupled = multicore/shared memory; loosely-coupled = clusters).

Match workload to parallelism: *Same operation on huge regular arrays* → SIMD/vector/GPU (the slides only mention these as "data-level parallelism"; modern graphics cards are SIMD). *Independent threads/requests* → MIMD: multicore + multithreading. *One sequential thread* → ILP hardware (pipeline, superscalar, speculation), but remember ILP saturates at roughly 3–6 issue.

Slides: ca01 "Parallelism", "Flynn's Taxonomy"; ca08 "General context: Multiprocessors"

2.3 The quantitative toolbox

Performance

$$\text{CPU time} = \text{IC} \times \text{CPI} \times T_{clk}, \qquad \text{CPI} = \frac{\sum_i \text{IC}_i \cdot \text{CPI}_i}{\text{IC}}$$

- **IC** (instruction count): ISA design and the compiler. **CPI**: microarchitecture (hazards, branches, memory stalls, issue width). T_{clk}: technology and pipeline depth.
- Every technique in this manual attacks one of these factors, and usually *trades* one against another (deeper pipeline: $T_{clk} \downarrow$ but CPI $\uparrow$).
- Speedup of X relative to Y = execution time$_Y$ / execution time$_X$. Benchmarks: kernels, toy programs, synthetic, suites (SPEC, TPC-C).

Amdahl's law:† if a fraction f of execution time is sped up by s, overall speedup $= 1/\big((1-f) + f/s\big)$. Slide principle: *focus on the common case.*

Slides: ca01 "Principles of Computer Design", "Measuring Performance"

Power and energy

Quantity	Formula / statement (slides)
Dynamic energy per switch	$\frac{1}{2} \times C_{load} \times V^2$
Dynamic power	$\frac{1}{2} \times C_{load} \times V^2 \times f_{switched}$
Static power	$I_{static} \times V$; scales with number of transistors; reduce by *power gating*
TDP (thermal design power)	Sustained power target for supply and cooling; lower than peak, higher than average
Better metric than power	Energy per task

- **Reducing the clock rate reduces power but not energy per task** (slide). Lowering *voltage* (DVFS) reduces energy because of the V^2 term.
- Techniques the slides list: dynamic voltage-frequency scaling, low-power states for DRAM/disks, turning off cores, overclocking within thermal limits, power gating (static power).
- Scale of the problem: Intel 80386 ≈2 W; 3.3 GHz Core i7 ≈130 W from a 1.5 cm×1.5 cm die, "the limit of what can be cooled by air".
- If voltage scales with frequency, dynamic power falls roughly with f^3 (0.8× clock ⇒ ≈0.51× power)†; this is why *two slower cores can beat one fast core* on throughput per watt.

Energy-first design: Choose DVFS/power gating, small caches, narrow issue and multicore of small cores whenever the scenario says battery, thermal envelope, or energy cost per request. Justify with $P \propto CV^2 f$ and "energy per task is the right metric".

Slides: ca01 "Power and Energy", "Dynamic Energy and Power", "Reducing Power", "Static Power"; ca06 "Limits to ILP"

Cost

- Cost falls along a *learning curve* (yield improves). DRAM price tracks cost; microprocessor price depends on volume (10% less per doubling of volume).
- Die yield follows the Bose–Einstein form yield = wafer yield × $(1 + \text{defects/area} \times \text{die area})^{-N}$ with N the process-complexity factor (slide: 11.5–15.5; defects 0.016–0.057/cm^2)† (formula is an image in the slide). **Consequence: bigger dies are disproportionately more expensive**, favouring several small cores over one huge core.

Slides: ca01 "Trends in Cost", "Integrated Circuit Cost"

Dependability

$$\text{MTBF} = \text{MTTF} + \text{MTTR}, \qquad \text{Availability} = \frac{\text{MTTF}}{\text{MTBF}}$$

For servers (availability is their emphasis) quote this and add redundancy and ECC (the slides mention error-correcting codes when discussing wide memory).

Slides: ca01 "Dependability"; ca05 "Wider Main Memory"

2.4 Technology trends that justify design choices

Trend (slide numbers)	What it implies for design
Bandwidth improved 10,000–25,000× for processors but only 300–1200× for memory/disks; latency improved 30–80× vs. 6–8×	Memory is the bottleneck ("memory wall"). Use caches, interleaving/wide memory, prefetching, and latency-hiding (OoO, non-blocking caches, multithreading). Bandwidth is easy to buy; latency is not.
Transistor density +35%/yr; transistor speed scales, **wire delay does not**	Huge centralised structures (many register ports, big CDB, wide issue logic) hurt clock rate. Favours smaller replicated cores and simpler structures.
Single-processor performance improvement ended in 2003; ILP cannot be leveraged further	Move to explicit parallelism: DLP, TLP, RLP ("require explicit restructuring of the application"). Justifies multicore.
DRAM capacity growth slowing (25–40%/yr); flash 15–20× cheaper per bit than DRAM; disk 15–25× cheaper than flash	Deeper hierarchy: SRAM caches → DRAM → flash → disk, each level sized by cost per bit.

Slides: ca01 "Trends in Technology", "Bandwidth and Latency", "Transistors and Wires", "Current Trends in Architecture"

3 Pipeline, Long-Latency Units and Branch Handling

3.1 Decision map

Problem observed	Feature to apply	Why it fits
Stalls from RAW hazards between dependent ALU ops	Forwarding/bypassing	Result is available at end of EX; forward instead of waiting for write-back.
Load followed by use	Compiler load-delay scheduling; forwarding from MEM	Load data is late; fill the slot(s) with independent instructions.
Control hazard, shallow pipe, predictable branches	Early PC evaluation in ID + static prediction / delay slot	Cheapest (no predictor memory), penalty already only 1 cycle.
Control hazard, deep pipe or high branch frequency	Dynamic prediction (BHT → correlating/two-level → gshare) + BTB	Penalty = several cycles; accuracy matters more than anything.
Loop-heavy code	BTFNT or 2-bit counter	Back-edge is taken almost always; 2-bit avoids the double misprediction.
Target address needed early	BTB (and target buffer / branch folding)	Fetch target in IF with 0–1 bubble.
Returns / indirect jumps	Return-address stack	Targets vary at run time; stack predicts them.
Divide-heavy or multiply-heavy FP code	Pipelined (or replicated) FP units	Initiation interval 25 for an unpipelined divider blocks every other divide.
Want higher clock	Deeper pipeline (R4000 8-stage)	Shorter stages; but raises load/branch delay.

3.2 Pipeline hazards: the three kinds and their standard cures

Hazard	Cause	Cure (slides)
Structural	Two instructions need the same hardware in the same cycle	More hardware resources (replicate/pipeline units, more ports).
Data (RAW/WAR/WAW)	Dependence between instructions	Forwarding; compiler scheduling; (hardware) register renaming for WAR/WAW.
Control	Next PC unknown until branch resolves	Early evaluation of PC, delayed branch, prediction.

- Pipelining improves instruction *bandwidth* (throughput), not the latency of one instruction.
- **Longer pipelines increase the impact of hazards**; interrupts, complex ISAs and FP all make pipelining harder.
- Compilers reduce the cost of data and control hazards: load delay slots, branch delay slots, branch prediction hints.

Slides: ca03_noor "Review: Summary of Pipelining Basics"

3.3 Long-latency functional units (FP pipeline)

The slides' running example is a RISC-V/MIPS-style pipeline in which EX may repeat, and there are several functional units.

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add (4-stage pipelined)	3	1
FP/integer multiply (7-stage pipelined)	6	1
FP/integer divide (not pipelined)	24	25

Latency = intervening cycles between the producer and a consumer that avoids a RAW stall. **Initiation interval** = cycles that must separate two operations of the *same* type issued to the unit.

Latency table used in the compiler-scheduling examples (cycles between producer and consumer): FP ALU→FP ALU 3; FP ALU→store 2; load→FP ALU 1; load→store 0; integer→integer 0.

- Longer pipeline ⇒ more frequent RAW stalls (e.g. the 4th instruction after an FP add is the first that can use its result without a stall).
- New hazards appear: **structural hazard at the register-file write port** (three instructions wanting write-back in the same cycle), **structural hazard on the unpipelined divider**, and **WAW hazards / out-of-order completion** because a short operation can finish before an earlier long one.

Pipelined vs. unpipelined divider: Pipelined divider (or several dividers) when the code has frequent divides close together: with initiation interval 25, a second divide stalls for up to 24 cycles. An unpipelined divider is a cheap, acceptable choice when divides are rare, since it needs far less area and power. Adder and multiplier are worth pipelining almost always (initiation interval 1).

Slides: ca03_noorNew "RISC V FP Operation", "Latencies and initiation intervals", "A pipeline that supports multiple outstanding FP operations"; ca03_noor "FP Loop: Where are the Hazards?"

3.4 Deeper pipelines: the MIPS R4000 case study

The R4000 splits the 5-stage integer pipeline into **8 stages**: IF, IS, RF, EX, DF, DS, TC, WB (cache access takes two stages each for instruction and data; TC is the tag check). The aim is a **higher clock rate**.

Effect	Value / reason
Load delay	**2 cycles** (the older deck states "TWO cycle load latency"; the newer deck's slide title reads "x1" but its pipeline table shows two Stall cycles — use 2). Data is available at end of DS and is bypassed; if TC reports a miss the pipeline backs up.
Branch delay	**3 cycles**: condition evaluated in EX; one delay slot plus two stalls; "branch likely" cancels the delay slot when not taken.
FP unit	Three units: divider, multiplier, adder; adder also used in the last step of multiply and divide. Stage kinds: U (unpack), S (operand shift), A (mantissa add), R (round), E (exception test), M, N (multiplier stages), D (divide stage, used 28 times in a row for divide).

Deep vs. shallow pipeline: *Go deeper* when the target is a high clock rate and the code has good branch predictability and few back-to-back load-use pairs (scientific/loop code), *and* you can afford a strong branch predictor. *Stay shallow* for branchy, low-ILP, low-power or area-limited designs: every extra stage adds to the misprediction and load penalty (and, in general, to power[†]) (deeper pipelines also magnify branch cost: slide "Branch Prediction Techniques").

Slides: ca03_noor "Case Study: MIPS R4000"; ca03_noorNew "The MIPS R4000 Pipeline" through "The Floating-Point Pipeline"

3.5 The control hazard problem in numbers

Where the branch resolves determines the penalty (5-stage MIPS)

Scheme	Stall cycles per branch
Resolve at MEM, no forwarding (conservative stall)	3
Resolve at EX (with forwarding)	2
Early evaluation: compare registers, compute target and update PC in ID	**1**

Early evaluation has a side-effect: an ALU instruction that produces the branch operand needs **1 extra stall**; a load that produces it needs **2 extra stalls**.

$$\text{stall cycles per instruction (branches)} = f_{branch} \times \text{penalty}; \qquad \text{CPI} = \text{CPI}_{base} + f_{branch} \times (\text{mispredict rate}) \times \text{penalty}^{\dagger}$$

Even a 1-cycle penalty costs 10–30% performance depending on branch frequency (slide). **Importance of prediction grows with (a) pipeline depth (penalty) and (b) branch frequency.**

Why prediction matters more in deep pipes. [†] $f_{branch} = 20\%$. Shallow pipe with early evaluation: always stall 1 cycle ⇒ +0.20 CPI. Deep pipe, penalty 3: always stall ⇒ +0.60 CPI; with a 90%-accurate predictor ⇒ $0.20 \times 0.10 \times 3 = +0.06$ CPI. The predictor recovers 90% of the loss.

Slides: ca04 "The Problem of Control Hazards" through "Branch Prediction Techniques"

3.6 Static branch prediction (decided at compile time)

Static schemes need no run-time hardware, so they are used where branch behaviour is highly predictable at compile time, and also to assist dynamic predictors.

Scheme	How it works	Best scenario	Weakness
Predict not taken	Keep fetching fall-through; flush 1 instruction if taken (1-cycle penalty)	Branches mostly not taken (error checks, forward branches); simplest hardware. If 50% not taken, halves control-hazard cost	Loops (back-edges are taken)
Predict taken	Fetch target once decoded	Only when the *target is known before the outcome* (needs BTA in IF or a BTB). No advantage in the base MIPS pipeline	Needs extra target hardware
BTFNT (backward taken, forward not taken)	Direction of branch decides	Loop-dominated code: back-edge branches almost always taken	Irregular forward branches
Profile-driven	Compiler uses profile from earlier runs/hints	Stable behaviour across inputs (fixed embedded workloads)	Behaviour that changes with input
Delayed branch	Compiler fills slot(s) after branch with a useful instruction; executed whether or not taken	Shallow pipe with a single slot; compiler fills ≈50% usefully	Deep pipelines (several slots, hard to fill); remaining slots become NOPs

Choosing where the delay-slot instruction comes from

Source	Use when	Legality
From before	Always valid if an independent instruction exists before the branch	Safe: executes the same either way
From target	**Branch usually taken** (loop back-edges); target instruction must be copied	Must be harmless (e.g. writes an unused temporary) if branch falls through
From fall-through	**Branch usually not taken** (forward branches)	Must be harmless if branch is taken

Canceling (nullifying) branch: the instruction encodes the predicted direction; if the prediction is wrong the delay-slot instruction becomes a NOP, so the compiler need not be conservative. MIPS *branch-likely* is cancel-if-not-taken (slot filled from target) — ideal for loop back-edges.

Slides: ca04 "Static Branch Prediction Techniques" through "Delayed Branch Technique" (all pages)

3.7 Dynamic branch prediction (decided by hardware at run time)

Two cooperating pieces: a **branch-outcome predictor** (taken?) and a **branch-target predictor/BTB** (where to?). The fetch unit uses them to pick the next PC in IF.

Branch history table (BHT) and counters

- **1-bit BHT:** table indexed by low-order bits of the branch PC; no tags (every access "hits"; aliasing is tolerated because the entry is only a hint). Weakness: a loop branch (taken 9, not-taken 1) mispredicts *twice* per loop execution (at exit, and on re-entry) ⇒ only **80% accuracy**.
- **2-bit saturating counter:** must be wrong twice before the prediction flips; loop exit costs only one misprediction. Predict taken if counter ≥ half of max.
- **n-bit counter:** generalisation; studies show 2-bit performs almost as well, so *use 2-bit*.
- A 4K-entry 2-bit BHT achieves 82–99% accuracy (1–18% misprediction) on SPEC89 and is almost as good as an infinite table; so beyond ∼4K entries, make the *predictor smarter*, not bigger. Aliasing is cured by more rows or a hash.

Correlating (two-level) predictors

Idea: the outcome of recent *other* branches helps predict this one (e.g. if the two preceding branches were both not taken, branch bb3 will be taken).

- (m, n) **predictor:** the last m branch outcomes (global history) select one of 2^m BHTs, each with n-bit counters. $(0, 2)$ is the ordinary 2-bit predictor. $(2, 2)$: four 2-bit BHTs selected by 2-bit history.
- Index = low-order PC bits concatenated with the m-bit history. Example: a $(2, 2)$ predictor with 64 total entries uses a 6-bit index = 2 history bits + 4 PC bits; each BHT has 16 entries.
- A small $(2, 2)$ predictor beats a plain 2-bit predictor of the same storage and often beats an unlimited-size 2-bit predictor.
- **Two-level adaptive:** first level = branch history register (BHR, a k-bit shift register of recent outcomes); second level = pattern history table (PHT) of 2-bit counters indexed by the BHR.
- **GAs:** PHT indexed by PC bits *and* global history (local + global). **gshare:** PHT indexed by **XOR** of low PC bits with the BHR (local XOR global), which uses the table better than concatenation[†].

Branch target buffer (BTB) and fetch-unit support

- BTB is a *cache* accessed in IF with the PC of the instruction being fetched. Entry: **tag + predicted target (PC-relative) + optional prediction bits**. Only **taken** branches need an entry. The tag *must* match (a wrong target cannot be used, unlike BHT aliasing).
- If BTB hit and predicted taken: fetch from the predicted PC immediately. On misprediction: kill fetched instruction, restart at the other target, update/delete the entry. Correct prediction: no stalls.

- **Target instruction buffers** store the target *instruction*(s): faster, allow a larger BTB, and permit **branch folding** (0-cycle unconditional branches).
- **Integrated instruction-fetch unit:** autonomous unit doing branch prediction, prefetch and instruction buffering to feed a wide back end.
- **Return-address predictor:** small stack of recent return addresses, for indirect jumps/returns whose destination varies at run time.

Which dynamic predictor for which scenario

Scenario	Choose	Reason
Small/cheap core, loop code	2-bit BHT	Fixes the loop double-miss at tiny cost (2 bits/entry, no tags).
Control-heavy integer code (compilers, interpreters), correlated if-chains	Correlating $(2,2)$ or two-level (GAs/gshare)	Uses other branches' outcomes; small table beats a much larger 2-bit table.
Same storage budget, want best accuracy	gshare	XOR indexing spreads history and PC across the table.
Wide-issue or deep pipeline	Two-level predictor + BTB + return-address stack, feeding *speculation*	Cost of a miss is large; wide issue may need a branch every cycle.
Many function calls/returns	Return-address stack	Returns are indirect with run-time targets.
Fixed embedded workload, no predictor area	Static: profile-driven or BTFNT (+delay slot)	No hardware; behaviour known offline.
Branch target needed in IF for 0–1 bubble taken branches	BTB (+ target buffer for folding)	Predicts the *address*, not just direction.

Branch-prediction traps:

- 1-bit loop example: accuracy is 80% for a 10-iteration loop (8 correct, 2 wrong), not 90%.
- "Predict taken" is useless unless the target address is available before the outcome (needs a BTB or early BTA).
- A BHT has no tags (hint only); a BTB *does* (an incorrect address cannot be used).
- Prediction alone is not enough for wide-issue machines: it must be combined with **speculation** (Section 4).

Slides: ca04 "Dynamic Branch Prediction Techniques" through "Branch Target Buffer", "High Performance Instruction Delivery"

4 Instruction-Level Parallelism: Scheduling, Speculation, Multiple Issue

4.1 Decision map

Scenario	Feature	Why
Regular loops, independent iterations, fixed known latencies	Loop unrolling + static scheduling (+ software pipelining)	Compiler creates independent instructions to fill latency slots and removes loop overhead.
Unknown dependences at compile time (memory aliasing, cache-miss latencies)	Dynamic scheduling (Tomasulo / scoreboard)	Hardware reorders at run time; compiler need not know the microarchitecture.
Few architectural registers, lots of WAR/WAW	Tomasulo (register renaming in reservation stations)	Renames in hardware to a larger virtual register set.
Simplest possible out-of-order, few non-pipelined FUs	Scoreboard (CDC 6600 style)	Cheap centralised control; accepts stalls on WAR/WAW and no forwarding.
Need precise exceptions + branch speculation	Tomasulo + reorder buffer (ROB)	Out-of-order execute, in-order commit; easy undo.
Need CPI < 1	Multiple issue: superscalar or VLIW	More than one instruction per cycle.
Binary compatibility across generations	Dynamically scheduled superscalar	Same binary, hardware finds ILP.
Fixed-product, compiler-controlled, power/area simplicity	VLIW (static scheduling)	No issue/hazard-detection hardware.
Wider than ∼4–6 issue is requested	Don't: go multicore/TLP instead	ILP saturates; energy inefficient; clock rate suffers.

4.2 Dependences: what limits ILP

Type	Meaning	Can it be removed?
True (data) dependence	Instruction j uses a result produced by i (RAW if a hazard)	**No.** It is the real data flow; instructions cannot execute in parallel.
Anti-dependence (name)	j writes a register/memory location that earlier i reads (WAR)	Yes: *register renaming* (just a naming conflict).
Output dependence (name)	i and j write the same location (WAW); order must be kept	Yes: register renaming.
Control dependence	Instruction executes only if a branch goes a certain way	Relaxed by speculation/prediction; must preserve exception behaviour and data flow.

- Register dependences are easy to detect (fixed names). **Memory dependences are hard**: does `100(R4)` equal `20(R6)`? Do different iterations touch the same address? (alias analysis).
- **Control rules for code motion:** an instruction control-dependent on a branch cannot be moved before it; an instruction not control-dependent on it cannot be moved after it.
- **Loop-carried dependence:** e.g. `A[i+1]=A[i]+C[i]` depends on the previous iteration, so iterations cannot run in parallel and the loop cannot be freely unrolled/parallelised. Safe to unroll when iterations are independent.

Slides: ca03_noor "Compiler Perspectives on Code Movement", "When Safe to Unroll Loop?"; lec06 "Review: Exceptions and Compiler Scheduling"

4.3 Static (compiler) scheduling

Loop unrolling: the slides' worked example

Loop: `LD F0,0(R1); ADDD F4,F0,F2; SD 0(R1),F4; SUBI R1,R1,8; BNEZ R1,Loop` (+ delay slot).

Version	Cycles	Cycles / iteration
Original (stalls after LD, ADDD, and in branch slot)	9	9
Reordered: SUBI moved up, store address adjusted (`SD 8(R1),F4`) to fill delay slot	6	6
Unrolled ×4, straightforward (still stalls)	$15 + 4 \times (1 + 2) = 27$	6.75
Unrolled ×4, scheduled with renamed registers (loads together, ADDDs together, stores together)	14	**3.5**

Why it works: loop overhead (SUBI, BNEZ) is amortised; independent instructions from different iterations fill the 1-, 2- and 3-cycle latency gaps. **What it requires:** (a) *renaming* (use F0/F4, F6/F8, ... instead of reusing F0/F4, otherwise name dependences stall you), (b) knowledge that `0(R1)`, `-8(R1)`, ... are different addresses so loads can move before stores, (c) no loop-carried dependence.

Software pipelining and static renaming

- **Loop unrolling:** multiple iterations in the loop body, all instructions from all iterations; needs compiler register renaming.
- **Software pipelining:** one instruction from each of several iterations in the body; turns dependences into anti-dependences; often used for *floating-point* with long latencies.
- Static renaming limits: dependence is only resolved within a window, and **increases register pressure** for the compiler.

Static scheduling / unrolling: Regular numeric loops with independent iterations and *known fixed latencies*; in-order or VLIW hardware; embedded/DSP-style kernels where simple hardware and predictable timing matter. Best when many registers are available.

Do NOT choose it when: Code with unknown memory aliasing, pointer chasing, data-dependent branches, or variable latency from cache misses; or when the same binary must run efficiently on future hardware with different latencies.

Slides: ca03_noor "FP Loop Hazards" through "Unrolled Loop That Minimizes Stalls"; lec06 "Review"

4.4 Dynamic scheduling: why do it in hardware

- Works when the dependence cannot be known at compile time; keeps the *compiler simple*; code tuned for one machine runs well on another.
- Key idea: **allow instructions behind a stalled one to proceed** (e.g. in `DIVD F0,F2,F4; ADDD F10,F0,F8; SUBD F12,F8,F14`, SUBD is independent and need not wait for the divide).
- Consequence: out-of-order execution ⇒ out-of-order completion ⇒ **WAR and WAW hazards** become possible, and **precise exceptions** become hard.
- Disadvantages: substantial hardware complexity; complicates exceptions.

Scoreboard (CDC 6600, 1963)

In-order issue, out-of-order execution and commit. Four stages replace ID/EX/WB:

1. **Issue:** functional unit free *and* no active instruction with the same destination (no WAW); otherwise stall, and nothing behind issues.
2. **Read operands:** wait until both operands are available (RAW resolved here); no forwarding.
3. **Execute.**
4. **Write result:** stall if a WAR hazard exists with an earlier instruction that has not yet read its operand.

Three tables: instruction status; functional-unit status (Busy, Op, Fi, Fj, Fk, Qj, Qk, Rj, Rk); register result status.

Limitations (slide): no forwarding hardware; limited to instructions in a basic block (small window, ≤ 5 instructions in comparison table); few functional units (structural hazards, especially integer/load-store); no issue on structural hazard; *waits* for WAR; *prevents* WAW.

Tomasulo's algorithm (IBM 360/91, 1966)

Goal: high performance *without special compilers*. The 360/91 had only **4 FP registers**, which blocked compiler scheduling; Tomasulo therefore **renamed registers in hardware**.

- **Distributed control:** *reservation stations (RS)* sit with each functional unit and hold the op, operand values V_j, V_k, and tags Q_j, Q_k (the RS that will produce the operand; $Q = 0$ means ready). Register specifiers are replaced by values or RS tags ⇒ **register renaming** that **eliminates WAR and WAW**. More RS than registers ⇒ optimisations compilers cannot do.
- **Common data bus (CDB):** results broadcast with their *source* tag ("come-from" bus: 64 data bits + 4 FU-id bits) to all RS, store buffers and the register file. Only the last writer updates the register.
- Loads and stores are treated as functional units with buffers (load/store buffers hold address and data); memory order is kept through effective-address computation.
- Three steps: **Issue** (if RS free, send operands/tags; else stall), **Execute** (when operands ready; watch the CDB otherwise), **Write result** (on CDB). No instruction starts until all earlier branches have resolved (non-speculative version).
- **Overlaps loop iterations:** renaming gives different iterations different destinations ("dynamic loop unrolling"); RS let integer instructions run ahead past branches so FP ops beyond a basic block can be queued; also hides cache misses.

Aspect	Tomasulo (IBM 360/91)	Scoreboard (CDC 6600)
Functional units	Pipelined: 6 load, 3 store, 3 add, 2 mult/div	Multiple, not necessarily pipelined: 1 load/store, 1 add, 2 mult, 1 div
Window size	≤ 14 instructions	≤ 5 instructions
Structural hazard	No issue	Same
WAR	Renaming avoids stall	Stall completion
WAW	Renaming avoids stall	Stall issue
Result delivery	Broadcast from FU on CDB	Write/read registers (no forwarding)
Control	Distributed (reservation stations)	Centralised scoreboard
Worked example (6-instr. sequence)	Last instruction writes in cycle 57	Same code finishes in cycle 62

Scoreboard is slower because of **structural hazards and lack of forwarding**.

Tomasulo drawbacks (slide): complexity; many associative stores (CDB) at high speed; **performance limited by the CDB** — each CDB goes to many units (high capacitance, wiring density), and only **one FU can complete per cycle per CDB**; more CDBs need more associative logic; **imprecise interrupts** (fixed by the ROB).

Hardware speculation: Tomasulo + reorder buffer (ROB)

Combines three ideas: **dynamic branch prediction** (choose what to run), **speculation** (execute before control dependences resolve, with ability to undo), **dynamic scheduling** (schedule across basic blocks). Core principle: **execute out of order, commit in order**; no irrevocable action (register/memory write, exception) before commit.

- **ROB:** circular buffer allocated in program order; entries hold instruction type (branch / store / register op), destination (register number or memory address), value, ready bit. It supplies operands between completion and commit and *replaces the store buffer*.
- **Four steps:** Issue (needs a free RS *and* ROB slot) → Execute → Write result (CDB → ROB) → **Commit** (at ROB head: write register/memory; a mispredicted branch at the head **flushes the ROB** and restarts at the correct successor).
- **Benefits:** *precise interrupts* (exceptions recorded in the ROB, recognised only at commit; a speculative instruction's exception is flushed with it); *early recovery* from mispredictions (clear entries after the branch, keep older ones); *memory hazards*: stores update memory only at the head, eliminating WAR/WAW through memory; RAW through memory is kept by not letting a load proceed if an earlier store has a matching address.
- **Costs:** large control complexity; associative network to find the latest version of a register (or a future file / register-status buffer); as many ROB ports as register-file ports; ROB sized to window.
- **Design questions:** register renaming (extended physical registers) *versus* ROB for holding values; **how much to speculate** — speculation is hurt by exceptional events (cache miss, TLB miss) and speculating through multiple branches complicates recovery.

Scoreboard vs. Tomasulo vs. Tomasulo + ROB:

- **No dynamic scheduling** (in-order + compiler): cheapest/lowest power; embedded, predictable loops.
- **Scoreboard:** want some out-of-order tolerance with minimal logic and few, non-pipelined units; accept WAR/WAW stalls.
- **Tomasulo:** FP-heavy code with long latencies, *few architectural registers*, or variable latencies (cache misses); you want renaming and loop overlap. Cap: one result per CDB per cycle.
- **Tomasulo + ROB (speculative):** high-performance desktop/server cores needing precise exceptions and branch speculation, especially with multiple issue. Most area/power.

Slides: ca03_noor ("HW Schemes", "Scoreboard", "Dynamic Scheduling - Tomasulo"); lec06 (whole deck: "Tomasulo vs. Scoreboard", "Tomasulo Drawbacks", "Four Steps of Speculative Tomasulo"); ca04 "Hardware-Based Speculation"

4.5 Multiple issue: superscalar vs. VLIW

To get CPI < 1 you must complete multiple instructions per clock. Three approaches: *statically scheduled superscalar*, *VLIW*, *dynamically scheduled superscalar*.

	Superscalar (dynamic)	VLIW (static)
Who finds parallelism	Hardware at run time (RAW: forwarding; WAR/WAW: renaming)	Compiler at compile time: bundles of K independent operations
Code size	Smaller	Larger: NOPs when bundles cannot be filled; unrolled loops
Binary compatibility	**Yes** across hardware generations	**Weak**: changing number of units ($K \to K+5$) or latencies breaks the schedule; fix = emulator, which hurts performance
Hardware	Complex issue/decode, dependence checking, renaming	**Simple decode/issue**, no interlock hardware (compiler checks)
Registers/ports	—	More registers; simplified; may need multiple register files; VLIW example needs 7 read/3 write int and 5 read/3 write FP ports
Stalls	Independent instructions continue	**Lock-step**: one hazard stalls the whole bundle
Examples	Pentium 4, Athlon 64, Power5	Itanium 2 (IA-64/EPIC)

Example VLIW word: one integer op (or branch), two independent FP ops, two independent memory references. There must be enough parallelism to fill the slots.

Dynamic multiple issue with speculation (modern microarchitecture): limit instruction classes per bundle (e.g. one FP, one int, one load, one store); examine dependences among the bundle's instructions and encode them in the reservation stations; need multiple completion/commit (multiple CDBs). **Issue logic becomes the bottleneck.**

Superscalar (dynamic): General-purpose products with many software generations, legacy binaries, irregular integer code and unpredictable latencies.

VLIW: Embedded/DSP or fixed-function products where the compiler is controlled, the code is regular (loops with abundant ILP), the hardware must be simple, and binary compatibility across generations is *not* needed.

Slides: ca03_noor "Static Instruction Scheduling" through "Dynamic Instruction Scheduling"; ca04 "Multiple Issue and Static Scheduling" through "Superscalar v. VLIW"

4.6 Limits of ILP (use these to argue *against* very wide issue)

Practical limits

- **1 branch in 5 instructions:** how do you keep a 5-wide machine busy?
- You need roughly **pipeline depth** × **number of FUs** independent instructions (e.g. $5 \times 4 = 15$–20) to keep the machine busy.
- Hardware: *easy* — more instruction bandwidth, duplicate FUs; *hard* — more register-file ports; *harder* — more memory ports; wide decode hurts clock rate/pipeline depth.
- Practical window sizes 32–126; larger windows are impractical (storage, comparisons, issue rate). Real processors are limited by number of FUs, buses, register ports.
- **Doubling issue from 3–6 to 6–12** would need 3–4 data memory accesses/cycle, 2–3 branches resolved/cycle, renaming/accessing > 20 registers/cycle, fetching 12–24 instructions/cycle ⇒ the **clock rate suffers** (the widest-issue processor, Itanium 2, has the slowest clock and the highest power).
- **Energy:** transistors switching ∝ peak issue rate, performance ∝ sustained rate; the gap grows, so *multiple-issue techniques are energy-inefficient.*

The ideal-processor study

Assumptions: infinite renaming registers (no WAR/WAW), perfect branch and jump prediction, perfect caches (all loads/stores 1 cycle), perfect memory-address alias analysis, unbounded window and issue. ILP is then limited *only by true data dependences.* Real model comparison: IBM Power5 issues 4/clk, 48 int + 40 FP rename registers, 2–6% misprediction, 64 KB I / 32 KB D / 1.92 MB L2 / 36 MB L3.

Head-to-head of four real ILP designs (Pentium 4 SMT, Athlon 64, Power5 SMT, Itanium 2 VLIW)

Efficiency rank (1=best)	Itanium 2	Pentium 4	Athlon 64	Power5
SPECInt / transistor	4	2	1	3
SPECFP / transistor	4	2	1	3
SPECInt / area	4	2	1	3
SPECFP / area	4	2	1	3
SPECInt / Watt	4	3	1	2
SPECFP / Watt	2	4	3	1

Take-aways (slide): no single leader; Athlon leads SPECInt; Itanium 2 and Power5 dominate on SPECFP; Itanium 2 is the least efficient on everything except SPECFP/Watt; Athlon and Pentium 4 use transistors and area well; Power5 is the best energy user on SPECFP and tied on SPECInt. Itanium (VLIW/EPIC) was "not a significant breakthrough in scaling ILP".

How wide should the core be?: Quote the slide conclusion: **3–6 issue is the practical limit**. Choose a core width of about 4 for a general-purpose design; go narrower for energy/area-limited designs; beyond that, spend transistors on *TLP* (multithreading, more cores). Right ILP/TLP balance is "decided in the marketplace": servers favour TLP, desktops may still prize single-thread speed.

Slides: ca04 "Limits to Multi-Issue Machines", "Studies of the Limitations of ILP"; ca06 (whole deck)

5 Memory Hierarchy: Caches, Main Memory, Virtual Memory

5.1 The framework: AMAT and the four levers

$$\text{AMAT} = t_{hit} + \text{miss rate} \times \text{miss penalty}, \qquad \text{CPI}_{total} = \text{CPI}_{base} + \frac{\text{mem. refs}}{\text{instr.}} \times \text{miss rate} \times \text{miss penalty}^{\dagger}$$

(The slides use exactly this CPI form in the memory-width example: $\text{CPI} = 2 + 1.2 \times 3\% \times 64 = 4.3$.)

Lever	Techniques (slides)
Reduce **miss penalty**	Multilevel caches (the most important), critical word first & early restart, read-miss priority over writes, merging write buffer, victim caches
Reduce **miss rate**	Larger blocks, larger caches, higher associativity, way prediction, pseudo-associative caches, compiler optimisations
Reduce penalty/rate via **parallelism**	Non-blocking caches, hardware prefetch, compiler-controlled prefetch
Reduce **hit time**	Small & simple caches, avoid address translation in indexing, pipelined cache access, trace caches

Why hit time matters most: it affects the clock cycle, so a fast hit helps *everything* and its importance is "multiplied" beyond the AMAT formula; also miss penalty (in cycles) depends on clock cycle time.

5.2 Step 1: diagnose the miss type (the "3 C's" + coherence)

Miss type	Definition	Reduced by	Does NOT fix
Compulsory (cold)	First access to a block; measured by an infinite cache; independent of cache size	Larger blocks (spatial locality), prefetching	Bigger cache or more associativity
Capacity	Cache cannot hold all blocks needed; measured by fully associative cache of same size; falls as cache grows	Larger cache; blocking/tiling (shrinks working set); better temporal locality	Associativity
Conflict (collision)	Too many blocks map to the same set; vanishes with full associativity; falls as associativity rises	Higher associativity, victim cache, pseudo-associative, way-predicted set-assoc., code/data layout	Compulsory/capacity
Coherence (multiprocessor)	Block invalidated by another processor's write. **True sharing:** the same word is written by another processor. **False sharing:** different words that happen to be in the same cache line	Data layout (separate independently-updated data into different lines)†; smaller blocks†	Bigger caches

Slide examples: 32 B lines, array of 32 doubles: first touch of `array[0]` is a cold miss; prefetching `array[1..3]` turns later accesses into hits. In a direct-mapped cache, `array[0]`→`array[16]`→`array[0]` conflicts even though the cache is not full. In a multiprocessor, P2 writing `array[1]` invalidates P1's line holding `array[0]`: false-sharing miss.

Slides: ca05 "Classify Cache Misses — 3 C's"; Cache-Misses ("Taxonomy of Cache Misses")

5.3 Reducing miss penalty

Multilevel caches ("probably the best miss-penalty reduction")

$$\text{AMAT} = t_{hit,L1} + m_{L1}\left(t_{hit,L2} + m_{L2}^{local} \times P_{L2}\right), \quad \text{global miss rate} = m_{L1} \times m_{L2}^{local}$$

- **Local** miss rate = misses in this cache / accesses *to this cache*. **Global** = misses / accesses generated *by the CPU*. **Global miss rate is what matters** when judging L2/L3.
- L1 capacity *and* conflict misses are served by L2, avoiding main memory. A big L2 ($\gg$L1) gives a global miss rate close to a single-level cache of that size; the slide's conclusion is that L2 *cache size* is what matters.
- **L2 associativity:** direct-mapped hit 10 cc, local miss 25%; 2-way 10.1→10 or 11 cc, local miss 20%; $P_{L2} = 100$ cc: DM $= 10 + 0.25 \times 100 = 35$; 2-way $= 10 + 0.20 \times 100 = 30$ (31 if the hit costs 11). **Higher associativity pays off at L2** because the extra hit time is tiny compared to the miss penalty.

Worked example. 1000 refs, 40 L1 misses, 20 L2 misses. $m_{L1} = 4\%$; local $m_{L2} = 20/40 = 50\%$; global $= 2\%$. With $t_{L1} = 1$, $t_{L2} = 10$, $P_{L2} = 100$, 1.5 refs/instr: $\text{AMAT} = 1 + 0.04(10 + 0.5 \times 100) = 3.4$ cc; stall cycles/instr $= (3.4 - 1) \times 1.5 = 3.6$.

Choose it when: Almost always. Essential when the processor is fast and memory is far (servers, desktops: L1+L2+L3). PMDs use two levels (64 KB L1 + 256 KB L2) to save energy/area. Hardware complexity 2 (costly if block sizes differ between levels).

Critical word first and early restart

Do not wait for the whole block. **Critical word first:** fetch the missed word first and send it to the CPU, then fill the rest of the block. **Early restart:** fetch in normal order but restart the CPU the moment the requested word arrives. **Value depends on block size and spatial locality:** useful mainly for *large blocks*; if the next access is the next sequential word in the block, the benefit shrinks.

Read-miss priority over writes; merging write buffer

- **Write-through with a write buffer:** a read miss may need data still sitting in the buffer (RAW through memory). Rather than waiting for the buffer to empty (increased read-miss penalty; one old MIPS 1000 design with a 4-word buffer saw about 50% worse), **check the buffer for an address match and let the read go ahead if there is none**.
- **Write-back:** on a read miss that replaces a dirty block, copy the dirty block to a buffer, do the read first, then write the dirty block: the CPU restarts sooner.
- **Merging write buffer:** an entry holds several words but a write is usually one word. Merge a new write into an existing valid entry with a matching address: fewer full-buffer stalls and faster multiword writes. Mainly used with write-through caches.

Victim cache

A small **fully-associative** cache between the cache and its refill path holding only blocks *just evicted*. On a miss, check it before going to the next level; if present, **swap** victim and cache block. No penalty increase because both are probed together.

- Jouppi: removes **20–95% of the misses** of a **4 KB direct-mapped** cache with 1–5 victim entries. AMD Athlon has 8 entries.

Choose it when: A direct-mapped (small, fast, simple) L1 suffering *conflict* misses — you keep the hit time of direct-mapped and recover much of 2-way's miss rate. Hardware complexity 2.

5.4 Reducing miss rate

Larger block size

Reduces compulsory misses (spatial locality) but **raises miss penalty** and, in small caches, conflict and capacity misses. Pick the block that minimises **AMAT**, not miss rate.

Block (B)	Penalty (cc)	4K	16K	64K	256K
16	82	8.027	4.231	2.673	1.894
32	84	**7.082**	3.411	2.134	1.588
64	88	7.160	**3.323**	**1.933**	**1.449**
128	96	8.469	3.659	1.979	1.470
256	112	11.651	4.685	2.288	1.549

AMAT (cc) vs. block size; memory model: 80 cc overhead + 2 cc per 16 B. **32–64 B blocks dominate**; a 256 B block only wins once the cache is 256 KB (miss rate vs. block table: 4 KB cache 256 B block 9.51% vs. 32 B 7.24%). Pentium 4's L2 uses 128 B blocks.

Block size: Small L1 → 32–64 B. Large L2/L3 → 64–128 B. Higher memory latency or bandwidth (overhead dominates the penalty) → somewhat larger blocks[†]. Small cache with 128/256 B blocks → wrong (conflict and capacity misses rise).

Larger caches and higher associativity

- **Larger cache:** helps capacity and conflict misses; costs hit time, area, power; popular for off-chip/L2.
- **Associativity:** 8-way is nearly as good as fully associative. **2:1 rule:** 2-way of size $N/2 \approx$ direct-mapped of size N (for caches <128 KB). Cost: longer hit time ⇒ clock cycle ×1.10 (2-way), ×1.12 (4-way), ×1.14 (8-way) relative to 1-way.

Size (KiB)	1-way	2-way	4-way	8-way
4	3.44	3.25	**3.22**	3.28
8	2.69	2.58	**2.55**	2.62
16	**2.23**	2.40	2.46	2.53
32	**2.06**	2.30	2.37	2.45
64	**1.92**	2.14	2.18	2.25
128	**1.52**	1.84	1.92	2.00
256	**1.32**	1.66	1.74	1.82
512	**1.20**	1.55	1.59	1.66

AMAT with the clock-stretch penalty included (the bold cell is the best per row; in the slide the *worse-than-left* cells are highlighted). **Reading:** for tiny (4–8 KB) caches 2–4 way wins; **from 16 KB upward direct-mapped has the lowest AMAT** once the slower clock is charged. This is the quantitative reason an L1 that sets the clock is small and low-associativity, while an L2 (off the critical path) can be highly associative.

Way prediction

Keep extra bits per set predicting which way will hit next; the multiplexer is set early and **only one tag comparison is done** first. A wrong prediction costs extra cycle(s) to check the remaining ways. Alpha 21264 uses it in its 2-way I-cache with $> 85\%$ accuracy (SPEC95). Predictor styles: last-used way, MRU, history table. Metrics: accuracy, misprediction rate, energy reduction.

Choose it when: Set-associative caches where **energy and hit time** matter: PMDs, instruction caches (sequential access makes prediction accurate), cores where a full parallel tag compare is too slow/hot. It keeps most of the associativity benefit while activating one way. Cost: predictor hardware, an extra access on a misprediction.

Pseudo-associative (column-associative) cache

Start as a **direct-mapped** cache; on a miss, probe a second location (invert the high-order index bit) **sequentially**, not in parallel. Gives fast hit (primary) and slow hit (alternate); swap blocks so the frequent one sits in the fast slot. **Problem:** a variable 1-or-2-cycle hit is hard for a CPU pipeline $\Rightarrow$ better for **caches not tied directly to the processor (L2)**; used in the L2 of the MIPS R10000 (and similar in UltraSPARC).

AMAT, slide values. 2 KB: 1-way 5.90, 2-way 4.90, pseudo 4.95. 128 KB: 1-way 1.50, 2-way 1.45, pseudo 1.371. Pseudo-associativity gets (nearly) 2-way miss rate at direct-mapped hit speed.

Compiler optimisations (zero hardware cost)

- **Code:** reorder procedures (profile-guided) to cut conflicts; McFarling: -50% misses for a 2 KB direct-mapped I-cache with 4 B blocks, -75% for 8 KB; optimised programs on a direct-mapped cache miss less than unoptimised on 8-way of the same size.
- **Merging arrays:** combine parallel arrays (`val[]`, `key[]`) into an array of structs $\Rightarrow$ better spatial locality, fewer conflicts.
- **Loop interchange:** change loop order so the inner loop walks memory sequentially (row-major). 1024×1024 ints, 64 B lines (16 ints): **3,145,728 $\rightarrow$ 196,608 misses for** $C = A + B$ **(-93.75%).**
- **Loop fusion:** merge loops over the same data so a value is reused while in cache (temporal locality). Example: A, B, C larger than the cache: 262,144 $\rightarrow$ 196,608 line transfers (-25%).
- **Blocking (tiling):** operate on $B \times B$ sub-matrices that fit in cache. Worst-case capacity misses $2N^3 + N^2 \rightarrow 2N^3/B + N^2$; also helps register allocation. Slide example: $N = 1024$, 32 KB cache, $B = 32$ (three tiles $= 3 \times 4$ KB$= 12$ KB fits): ≈ 1.07 billion misses $\rightarrow \approx 2.1$ million (simplified model).

Choose it when: Array/loop-dominated code with large data sets (scientific, image, matrix); embedded systems where adding hardware is impossible. *Matrix multiply with arrays accessed both by rows and columns: interchange does not help $\Rightarrow$ use blocking.*

Do NOT choose it when: Pointer-chasing, irregular data structures (no static pattern); loops where interchange/fusion is illegal because of dependences.

Slides: ca05 "Reducing Miss Rate" through "Summary of Compiler Optimizations", "Way Prediction", "Pseudo-Associative Caches"; CachePerformance2 (way prediction, pseudo-associative, loop interchange/fusion/blocking numerics)

5.5 Reducing penalty/rate through parallelism

Non-blocking (lockup-free) cache

Cache continues to supply hits during a miss. **Hit-under-miss** (one outstanding miss), **hit-under-multiple-miss / miss-under-miss** (overlap several misses; Pentium Pro allows 4). **Requires an out-of-order CPU** (scoreboard/Tomasulo) and **multiple memory banks**; greatly increases the cache controller complexity; must detect repeat misses to a block already being fetched.

- OoO CPUs can hide L1 misses that hit in L2, but **not a significant fraction of L2 misses**.
- Example (8 KB, FP): miss rate$\times$penalty: DM $11.4\% \times 16 = 1.84$; 2-way $10.7\% \times 16 = 1.71$. For integer: DM $7.4\% \times 16 = 1.18$, 2-way $6.0\% \times 16 = 0.96$. Hit-under-miss does **not increase hit time**, unlike associativity.
- Hard to evaluate: effective miss penalty is the *non-overlapped* time the CPU stalls.

Choose it when: Any out-of-order superscalar, especially memory-intensive FP code; pair with banked/interleaved memory. Useless on a strict in-order blocking CPU.

Hardware prefetching (stream buffers)

Fetch what you expect to need before it is requested, outside the cache. Alpha AXP 21064: on an I-miss fetch 2 blocks (target to I-cache, next to the **instruction stream buffer**); 1/4/16-block ISB catches 15–25%/50%/72% of misses. Data: one data stream buffer removed 25% of a 4 KB cache's misses, 4 streams 43%; 8 streams 50–70% (for two 64 KB 4-way caches, scientific programs). **Needs spare memory bandwidth** (otherwise it hurts).

Slide. Hit time 2 cc, prefetch-hit time 1 cc, prefetch-hit rate 25%, miss rate 1.10% (8 KB), miss penalty 50 cc:
AMAT $= 2 + 0.011 \times 0.25 \times 1 + 0.011 \times 0.75 \times 50 = 2.41525$ cc. A cache without prefetching needs a miss rate of only 0.83% (between 8 KB at 1.10% and 16 KB at 0.64%) to match.

Compiler-controlled prefetching

Special prefetch instructions (register prefetch: HP PA-RISC; cache prefetch: MIPS IV, PowerPC, SPARC v9), non-faulting (a form of speculative execution), best for loops (e.g. `prefetch(b[j+7][0])`). Cost: issuing prefetches takes instruction slots, so benefit must exceed issue cost; needs a non-blocking cache. Works for instructions too.

Hardware vs. compiler prefetch: Hardware (stream buffers): sequential/streaming access, no compiler support, spare bandwidth. Compiler: regular strided loops where you know the distance; embedded systems with controllable code. Skip prefetching on bandwidth-starved or energy-limited designs.

Slides: ca05 "Reducing Cache Miss Penalty or Miss Rate Via Parallelism" through "Compiler-Controlled Prefetching"

5.6 Reducing hit time

- **Small and simple caches:** the slow part of a hit is index → read tag → compare. Keep L1 small enough to be on-chip; direct-mapped is simpler/faster (more misses but faster hit); for L2 keep tags on-chip and data off-chip. Used when the cache sets the clock cycle.
- **Avoid address translation during indexing.** *Physically addressed* caches translate with the TLB first; a *virtually addressed* cache indexes in parallel with translation. Problems of pure virtual caches:
  - **Protection** bits must be copied on miss and checked every access.
  - **Process switch** ⇒ same virtual address ≠ same physical data ⇒ flush (huge task-switch cost and compulsory misses); fix: **PID in the tag** (PIDs save 0.6–4.3% miss rate over purging; cost 0.3–0.5% for uniprocess).
  - **Synonyms/aliases:** two virtual addresses for one physical address ⇒ two copies; fix: hardware anti-aliasing (check PA on miss, invalidate), or **page colouring** (aliases share low-order bits, e.g. 18 bits).
  - **I/O** uses physical addresses; needs mapping to VA.

  Compromise from the slides: overlap cache access with VA translation, requiring the cache index to remain invariant across translation (trivial if the cache is small); used in Alpha 21164 and UltraSPARC III.
- **Pipelined cache access / pipelined writes:** write hits normally need tag check before data write; pipeline into two stages so the *current write's tag check overlaps the previous write's data update*; looks like one write per cycle, short cycle time. Works if the CPU does not immediately depend on the written data (watch read/write ordering). Reads are unaffected (tag check in parallel with data read).
- **Trace cache:** stores dynamic instruction *traces* from branch entry to exit, avoiding wasted space in conventional blocks (a taken branch every 5–10 instructions means much of a 16-instruction/64 B block is unused). Costs: complicated address mapping, duplicate storage of the same instructions in different traces. Used in Pentium 4 (NetBurst).

Choose it when: Small simple L1 / pipelined writes: every high-clock design. Virtual/overlapped translation: tight L1 designs with a small cache. Trace cache: wide-issue front ends that need fetch bandwidth across taken branches (not energy-constrained designs).

Slides: ca05 "Reducing Hit Time" through "Trace Caches (Cont.)"

5.7 Cache optimisation summary table (from the slides)

Technique	Miss pen.	Miss rate	Hit time	HW compl.	Comment
Multilevel caches	+			2	Costly; widely used
Critical word first / early restart	+			2	Widely used
Read priority over writes	+			1	Trivial for uniprocessor
Merging write buffer	+			1	Used with write-through
Victim caches	+	+		2	AMD Athlon: 8 entries
Larger block size	−	+		0	Trivial; Pentium 4 L2 128 B
Larger cache size		+	−	1	Widely used, esp. L2
Higher associativity		+	−	1	Widely used
Way-predicting caches			+	2	UltraSPARC III I-cache
Pseudo-associative		+		2	MIPS R10000 L2
Compiler techniques (miss rate)		+		0	Software is the challenge
Non-blocking caches	+			3	All out-of-order CPUs
HW prefetch (instr./data)	+	+		2/3	UltraSPARC III
Compiler-controlled prefetch	+	+		3	Needs non-blocking cache
Small and simple caches			+ (− miss rate)	0	Trivial; widely used
Avoid translation in indexing			+	2	Trivial if small; Alpha 21164
Pipelined cache access			+	1	Widely used
Trace cache			+	3	Pentium 4

(+ improves, − hurts; HW complexity 0 = none, 3 = highest.) Note the *conflicts of interest*: larger blocks hurt penalty; larger size/associativity hurt hit time; small/simple caches hurt miss rate.

5.8 Main memory

DRAM vs. SRAM and key metrics

- **Main memory is DRAM:** dynamic, needs periodic refresh (about 8 ms, ∼1% of time); address split into RAS and CAS halves (2-D array). **Caches are SRAM:** no refresh; 6 transistors/bit vs. 1. DRAM/SRAM size ratio 4–8; SRAM/DRAM cost and cycle-time ratio 8–16.
- **Latency:** access time (request → word arrives) and cycle time (minimum between requests, > access time) ⇒ miss penalty. **Bandwidth:** bytes per unit time ⇒ time to transfer the block (matters for I/O and large-block L2 misses).
- DDR trend (slide tables): total best-case access time improved only from 42 ns (DDR1, 2000) to 26 ns (DDR4, 2016), whereas DIMM bandwidth rose from 2128 (PC2100) to 21,300 MiB/s (PC21300) — again **bandwidth far outgrows latency**.

Three ways to raise memory bandwidth

Organisation	What it is	Pros	Cons
Simple	CPU, cache, bus, memory all one word wide	Cheapest	Block transfer is serial: penalty ∝ words per block
Wide	Memory/bus (and cache side) N words wide (Alpha 64/256 bit, UltraSPARC 512 bit)	Bandwidth $\times N$; penalty drops accordingly	Wider bus cost; MUX between cache and CPU can be on critical path (put MUX between L1 and L2); ECC harder (partial update needs read of the rest); expansion in larger increments
Interleaved	Same width bus, N memory banks; bank = address mod #banks	Overlaps bank accesses, shares bus; great for **sequential** access (cache blocks); cheap	Needs ≥ (access time in cc) banks to deliver one word/cycle; many small chips costly; hard to expand
Independent banks	Multiple controllers	Multiprocessor/I/O, non-blocking caches (hit under n misses)	Controller complexity

Slide: what interleaving/width buys. Base CPI 2, 1.2 mem. refs/instr, miss penalty = 4 (address) + 56 (access) + 4 per word. Miss rates: 3%/2%/1.2% for 1/2/4-word blocks.

Configuration	1-word	2-word	4-word
64-bit bus & memory, no interleaving	4.30	5.07	5.69
64-bit bus & memory, interleaved	—	3.63	**3.09**
128-bit bus & memory, no interleaving	—	**3.54**	3.84

Note: the slide labels the narrow bus "64-bit" and the wide bus "128-bit"; its arithmetic treats narrow as one word per transfer and wide as two words per transfer. **Reading:** wide/interleaved beat simple by a large margin; with larger blocks **interleaving** wins (it overlaps accesses on a narrow bus), with 2-word blocks a **128-bit** bus is marginally better. If the bus cannot be widened (pins/cost), choose interleaving.

Slides: ca05 "Main Memory" through "Independent Memory Banks"

Memory hierarchy by device class (slide figures)

PMD: L1 64 KB (1 ns), L2 256 KB (5–10 ns), no L3, 1–2 GB DRAM (50–100 ns), flash 4–64 GB. **Laptop/desktop:** adds L3 (4–8 / 8–32 MB, 10–20 ns), 4–64 GB DRAM, flash. **Server:** L3 16–64 MB, 32–256 GB DRAM, flash 1–16 TB and disk 16–64 TB (5–10 ms).

5.9 Virtual memory

- **Why:** many processes each with their own address space share a smaller physical memory; need *protection* (a process only touches its own blocks); faster program start (not everything resident); frees the programmer from overlays.
- **Pages vs. segments:** pages = fixed-size blocks (4–8 KB), one address (page number + offset), **no external fragmentation**, allocation anywhere (non-contiguous), no compaction; segments = variable size (1 B up to 2^{16}–2^{32} B), address needs 2 words (segment number + offset). **Paging is the default choice**.
- **Translation:** virtual page number indexes the **page table** (valid bit, physical page number); valid = 0 ⇒ **page fault** (OS loads from disk). The page table is in memory, so every access would cost two memory accesses ⇒ **TLB**, a special cache of translations.
- **TLB:** usually small (≤ 128–256 entries); high-end: fully associative; mid-range: small n-way set associative. Fields: valid, dirty, tag, physical page number. Physical address then drives cache tag compare (TLB → cache interaction).
- Example split: 32-bit VA = 20-bit VPN + 12-bit offset (4 KB pages).

Slides: ca05 "Virtual Memory" through "Techniques for Fast Address Translation"

5.10 Scenario → memory design, quick reference

Symptom / constraint	Choose	Reason
Small direct-mapped L1, ping-pong conflicts	Victim cache; or pseudo-assoc./2-way; or array merging/padding	Conflict misses; victim cache keeps DM hit time.
Streaming/sequential large arrays	Stream-buffer prefetch, moderate block (64 B), interleaved memory, loop interchange	Compulsory/spatial misses; bandwidth is plentiful.
Large matrix multiply	Blocking (tiling) with 3 tiles $\leq$ cache	Capacity misses; interchange cannot help both row/column accesses.
OoO core with many outstanding loads	Non-blocking cache + banked memory	Overlap misses; otherwise OoO core is wasted.
L1 hit time limits clock	Small, simple (DM) L1, pipelined writes, VA/translation overlap	Hit time is multiplied across every instruction.
PMD / battery	Two cache levels, way prediction, moderate sizes, no trace cache, selective prefetch	Energy and area; way prediction cuts tag-compare energy.
Server with large working set	L3 (16–64 MB), high-associativity L2, interleaved/banked memory	Global miss rate matters; capacity misses dominate.
Write-through L1	Merging write buffer + read priority over writes	Prevent buffer-full stalls and RAW delays.
Large blocks and long block fill	Critical word first & early restart	CPU resumes when the needed word arrives.
Instruction-fetch bound, taken branches every 5–10 instr.	Trace cache or I-stream buffer; BTB	Fetch bandwidth across taken branches.
Multicore sharing data	Watch coherence (true/false sharing) misses; separate independently-written data into different lines†	Coherence misses are a distinct fourth C.
Cache must be tiny but fast	Direct-mapped, small block	Table: DM wins AMAT $\geq$16 KB; for 4–8 KB use 2–4 way.

6 Thread-Level Parallelism: Multithreading, SMT, Multicore

6.1 Decision map

Scenario	Feature	Why
Throughput server, many independent threads/requests	Multicore (CMP) with multithreading per core	Applications with TLP (DB, web) map directly; ILP alone is saturated.
Pipeline stalls are *long and rare* (e.g. L2 misses), want no slowdown of a single thread	Coarse-grained multithreading	Switch only on costly stalls; pipeline refill $\ll$ stall time.
Stalls are *short and frequent*; hardware can switch every cycle	Fine-grained multithreading	Hides short and long stalls; interleaves each cycle.
Out-of-order superscalar with idle issue slots; want cheap extra throughput	SMT	Reuses renaming and OoO hardware; fills unused slots from another thread.
Single important thread must run as fast as possible	One big (SMT-capable) superscalar core; preferred-thread priority	Multi-core does not speed one thread; SMT with priority protects it.
Limited die area / thermal budget but threaded software	Several small cores	Each core simpler and easier to design/manufacture; better energy per throughput.
Applications hard to parallelise	Do *not* rely on cores/threads	TLP gives no gain; spend on ILP/caches.

6.2 Why TLP at all

- ILP gives implicit parallelism within a loop or basic block; it saturates at 3–6 issue (Section 4), and for most applications **most execution units sit idle** even on an 8-way superscalar (Tullsen et al.).
- **TLP is explicit:** multiple threads, each with its own instructions, data, PC and register state. Goals: improve *throughput* of machines running many programs and *execution time* of multithreaded programs. TLP "could be more cost-effective to exploit than ILP".
- Natural parallelism: online transaction systems (many independent queries/updates), data mining, databases, scientific codes; server-per-client threads; games (AI, graphics, physics as threads).
- Industry move: IBM Power4 (2000) was the first commercial single-chip general-purpose multiprocessor (2 Power3 + shared L2); Sun, AMD, Intel followed.

Slides: ca06 "Performance beyond single thread ILP", "Thread Level Parallelism (TLP)"; ca08 "Why multi-core?", "Thread-level parallelism"

6.3 Multithreading on one core

Multiple threads **share the functional units** of one processor by overlapping. The processor *duplicates the independent state of each thread*: a separate register file, a separate PC, and (for independent programs) a separate page table. Memory is shared via the virtual-memory mechanism. Hardware thread switch is *much faster* than a full process switch (100s–1000s of clocks).

	Fine-grained	Coarse-grained	SMT (simultaneous)
When it switches	Every instruction/clock, round-robin, skipping stalled threads	Only on costly stalls (e.g. L2 miss)	No switching: instructions from several threads issue in the *same cycle*
Hides	Short *and* long stalls	Only long stalls (pipeline refill $\ll$ stall time)	Short and long stalls *and* wasted issue slots
Single-thread cost	Slows a ready thread (delayed by others)	Does not slow a thread (others run only when it stalls)	Can slow a thread unless priority is given
Hardware	Switch every cycle; per-thread state	No very fast switch needed; pipeline must be flushed/frozen on switch	OoO superscalar + per-thread rename table, PC, (logically) per-thread ROB
Example	Sun Niagara	IBM AS/400	Pentium 4 (HT), IBM Power5

Pick the multithreading style:

- **Coarse-grained** if the core is simple/in-order, stalls are dominated by rare, long events (L2/L3 misses), and single-thread speed must not degrade; avoid when short stalls dominate (refill cost cannot be recovered).
- **Fine-grained** for throughput-oriented cores with many threads where single-thread latency is secondary (server/web/DB); needs per-cycle switching hardware.
- **SMT** when you already have a wide out-of-order superscalar (renaming, large physical register set, OoO completion): *the marginal hardware is small.* Not worthwhile for narrow in-order cores.

Why SMT is cheap on an OoO core

Dynamically scheduled processors already have: a large set of virtual (physical) registers that can hold several threads' register sets; **register renaming** gives unique register ids so instructions from different threads mix in the datapath without confusing sources and destinations; **out-of-order completion** lets threads proceed independently. You only add a per-thread renaming table and separate PCs; independent commit is obtained by logically keeping a separate ROB per thread.

SMT design challenges and costs (IBM Power5 case study)

- **Single-thread performance:** SMT needs fine-grained scheduling; a **preferred-thread** approach protects one thread but loses some throughput when that thread stalls. Power5 gives hardware-controllable *relative thread priority*; with balanced threads both run slower than if each owned the machine.
- Larger register file for multiple contexts; do not lengthen the clock cycle (more candidate instructions to issue, harder choice at commit); avoid cache/TLB conflicts between threads.
- **Why only 2 threads in Power5?** With 4, one of the shared resources (physical registers, cache, memory bandwidth) would become a bottleneck.
- **Changes made in Power5 to support SMT:** higher associativity in the L1 I-cache and instruction translation buffers; per-thread load/store queues; larger L2 (1.92 vs. 1.44 MB) and L3; separate instruction prefetch and buffering per thread; physical registers 152 $\rightarrow$ 240; larger issue queues. **Power5 core is ≈24% larger than Power4's** because of SMT.

SMT performance data to quote

Measurement	Result (slides)
Pentium 4 Extreme SMT, SPECrate (same benchmark $\times$ copies)	1.01 (SPECint_rate), 1.07 (SPECfp_rate)
Pentium 4, 26 SPEC benchmarks paired with each other (262 runs)	Speedups 0.90–1.58, **average 1.20**
Power5, 8-processor server	1.23 (SPECint_rate), 1.16 (SPECfp_rate) with SMT
Power5, two copies of each application	0.89–1.41; most gained; **FP applications had the most cache conflicts and the least gains**
General statement (multicore deck)	"Enables better threading (e.g. up to 30%)"

Reading: SMT gives a modest, workload-dependent gain (roughly 1.0–1.4$\times$) and can *hurt* (speedup <1) when threads fight over caches. It is *not* a true parallel processor: there is a single copy of each resource, and threads cannot use the same functional unit simultaneously.

Area-efficiency argument for SMT vs. a second core. †Power5 SMT cost ≈24% extra core area for ≈16–23% throughput. A second core costs about 100% extra core area but can give up to ∼2$\times$ throughput for fully parallel software. So SMT is roughly *area-neutral* on throughput but needs little extra design and shines when threads are stall-bound; a second core wins when software is truly parallel and area/power allow.

Slides: ca06 "New Approach: Multithreaded Execution" through "Initial Performance of SMT"; ca08 "Simultaneous multithreading (SMT)"

6.4 Multicore (chip multiprocessor, CMP)

- Several cores on one processor socket; they run in parallel; **within each core threads are time-sliced** as on a uniprocessor. The OS sees each core as a separate processor and schedules threads/processes across them.
- **Why multicore:** single-core clock frequencies cannot rise further; limited ILP; many applications are already multithreaded; general trend toward parallelism.

- A multicore is a **shared-memory MIMD multiprocessor on one chip**: different cores run different instruction streams on different data, all sharing the same memory.
- **Applications that benefit:** database servers, web servers, compilers, multimedia, scientific, CAD/CAM; "anything that can be threaded". Some applications are difficult to parallelise.

Multicore vs. SMT, and combining them

	Multicore	SMT
Resources	Each core has its own copy of resources	Single copy of each resource shared by threads
Core size/design	Several smaller, less powerful cores; easier to design and manufacture	One large, fast superscalar core
Strength	Great with thread-level parallelism; independent execution	Great single-thread performance; still mostly ILP-based
Weakness	Does nothing for one thread; coherence traffic	Not true parallelism; shared-resource contention; $\leq$30% gain
Threads	One (or more) per core	2, 4 or sometimes 8 per core (Intel "hyper-threads")

Four combinations: single-core non-SMT (standard uniprocessor), single-core SMT, multi-core non-SMT, **multi-core with SMT** (the best-throughput option; e.g. SMT dual-core runs four threads concurrently).

Memory hierarchy in threaded designs

- SMT only $\Rightarrow$ all caches shared. Multicore $\Rightarrow$ **L1 caches private**; L2 **private in some designs, shared in others**; **main memory always shared**. Example: dual-core hyper-threaded Xeon with private L1 and **shared** L2.
- **Shared L2:** threads share capacity dynamically and share data without duplication†; risk of interference. **Private L2:** isolation and simple timing†; duplicated data, wasted capacity if load is unbalanced†. In either case sharing introduces **coherence misses** (true/false sharing; Section on miss types).

Multicore + SMT: Server/throughput workloads and mixed multitasking desktops: plenty of independent threads, area available, TLP-friendly software. Add SMT to each core if the core is a wide OoO and memory stalls are common.

Do NOT choose it when: Single-threaded or hard-to-parallelise applications (give one fast core instead); extremely tight power budgets that cannot afford duplicate structures or shared-cache contention.

ILP vs. TLP balance (slide conclusion): There is no universal answer: *server* market $\Rightarrow$ more TLP (many cores/threads); *desktop* where single-thread speed is still a primary requirement $\Rightarrow$ keep a strong ILP core but add a few cores. "Balance of ILP and TLP decided in marketplace."

TLP traps:

- SMT threads cannot use the *same* functional unit at the same time; it is not a doubling of resources.
- Coarse-grained multithreading is useless when stalls are short (pipeline refill cost dominates); fine-grained slows individual threads.
- More threads $\neq$ more speed: Power5 stops at 2 threads because shared resources saturate; SMT can give speedup < 1 on cache-conflict-heavy (FP) code.
- Multicore does not accelerate a single thread; the OS must have threads to schedule.
- False sharing causes coherence misses even though data is not shared.

Slides: ca08 (multicore deck: "Multi-core architectures" through "Designs with private L2 caches"); ca06 "Summary", "Conclusions"; Cache-Misses (coherence misses)

7 Putting It Together: Scenarios, Dependencies, Checklist

7.1 Which features only work together (check your design for consistency)

Exam answers lose marks for choosing a feature whose prerequisite is missing. Use this table as a consistency check.

If you choose...	You also need...	Otherwise...
Non-blocking cache (hit-under-miss)	Out-of-order CPU (scoreboard/Tomasulo), multiple memory banks	Blocking CPU stalls anyway; cannot support multiple outstanding misses
Hardware speculation	Dynamic branch prediction, reorder buffer (commit in order), recovery/flush	No way to undo wrong-path work; imprecise exceptions
Wide issue (superscalar)	Accurate predictor + BTB, more register-file/memory ports, renaming, enough independent instructions	Issue slots idle; clock rate and power suffer
SMT	Out-of-order superscalar core, renaming, larger physical register file, per-thread PC/rename/ROB, cache associativity/size increase	Needless on narrow in-order core; shared-resource conflicts

If you choose...	You also need...	Otherwise...
Predict-taken static scheme	Branch target known early (BTB/early BTA)	No benefit
VLIW	Strong compiler (alias analysis, unrolling, scheduling), no need for binary compatibility	Poor code size/utilisation, lock-step stalls
Hardware prefetch	Spare memory bandwidth	Prefetches compete with demand misses
Compiler prefetch	Non-blocking cache; issue slots	Wasted instructions; stalls
Way prediction / pseudo-associative	Set/column structure; tolerance for variable hit latency (better for L2 than L1 for pseudo)	Pipeline problems when hit time varies
Virtual-address cache	PID tags, anti-aliasing (page colouring), protection bits	Flushes on context switch; synonyms

7.2 Reusable full-answer skeleton

1. **Class and goal:** (e.g. "PMD: energy-efficient, real-time").
2. **Workload analysis:** parallelism type, branch behaviour, memory behaviour, latency tolerance.
3. **Core:** pipeline depth, issue width, scheduling (static/dynamic/speculative), why.
4. **Control:** branch predictor type and size, BTB, return stack.
5. **Memory:** L1 (size, associativity, block, hit time), L2/L3, miss-penalty/miss-rate/hit-time techniques, main memory organisation, TLB.
6. **Parallelism beyond ILP:** threads per core, number of cores.
7. **Quantify at least one claim** using AMAT / CPI / stalls-per-instruction.
8. **Costs and rejected alternatives.**

7.3 Worked scenarios (model answers)

Scenario 1 — Battery-powered smartphone SoC

Constraints: battery life, thermal limit, real-time UI/media, low cost, mixed integer code, occasional bursts.

Block	Choice	Why
Class/goal	PMD: energy efficiency + real-time	Slide emphasis. Optimise energy per task, not clock.
Core	Narrow (2-way) in-order or modest OoO core, shallow pipeline (5–8 stages)	ILP techniques are energy-inefficient (transistors switch with peak issue rate, performance follows sustained rate); deep pipelines magnify branch penalty.
Branch	Small 2-bit BHT (+ small BTB, return stack) or BTFNT/static	Cheap and accurate on loops; avoids large predictor area/energy.
L1	32–64 KB, 2-way with **way prediction**, 32–64 B blocks	Way prediction does one tag comparison (energy); 32–64 B minimise AMAT.
L2	256 KB, no L3	Slide PMD hierarchy: L1 64 KB → L2 256 KB → 1–2 GB DRAM → flash.
Memory	Interleaved (or modestly wide) DRAM; low-power DRAM states	Bandwidth at low cost; DRAM low-power states listed in slides.
Power	DVFS, power gating, turn off idle cores	Dynamic power $\frac{1}{2}CV^2f$; static power cut by gating.
Parallelism	Several small cores (big.LITTLE-like)†; **no SMT**	Small cores are easy to build; SMT adds area/power for ∼20%.

Rejected: wide speculative core (power), trace cache (area/energy), large L3 (not in PMD hierarchy), VLIW (binary compatibility for apps).

Scenario 2 — High-throughput web/database server

Constraints: thousands of independent client requests, availability 24/7, memory-bound database queries, cost per request.

Block	Choice	Why
Class/goal	Server: availability, scalability, throughput	Request-level parallelism: threads are independent.
Cores	**Many cores (CMP) + 2–4 SMT/fine-grained threads per core**	TLP is abundant; ILP of one thread is limited (most units idle). DB/web are listed as the best multi-core beneficiaries.
Core style	Moderate-width (4-issue) OoO with speculation, or simpler fine-grained cores	Cores are smaller/easier to build; throughput over latency.
Multithreading	Fine-grained or SMT	Hides short and long (DRAM) stalls; DB workloads are memory-bound.
L1/L2/L3	Private L1; L2 private or shared; **large shared L3 (16–64 MB)**	Slide server hierarchy; global miss rate matters; high-associativity L2 is cost-effective.
Memory	Large DRAM (32–256 GB), **interleaved/independent banks**, non-blocking caches	Many outstanding misses from many threads; banks support multiprocessor traffic.

Block	Choice	Why
Dependability	ECC memory, redundancy; availability = MTTF/(MTTF+MTTR)	Server emphasis on availability.
Coherence	Aware of true/false sharing	Cores share data; coherence misses appear.

Rejected: very wide single core (ILP saturates), coarse-grained MT only (short stalls), aggressive deep speculation burning power per request.

Scenario 3 — Desktop/gaming: single-thread speed is the priority

Constraints: one dominant thread, integer + some FP, branchy code, price-performance.

Block	Choice	Why
Core	**4-wide dynamically scheduled speculative (Tomasulo + ROB)**, moderate-deep pipeline	Needs ILP and precise exceptions; 3–6 issue is the practical sweet spot (Athlon led SPECInt with 3-issue).
Branch	**Two-level / gshare predictor + BTB + return-address stack**, integrated fetch unit	Branchy code; deep pipe ⇒ misprediction cost high.
L1	Small (32–64 KB), low associativity, pipelined writes, 64 B block	Hit time sets the clock; DM/low-way wins AMAT ≥16 KB.
L2/L3	256 KB L2 (higher associativity) + 8–32 MB L3	Desktop hierarchy; L2 associativity cheap relative to penalty.
Latency hiding	Non-blocking cache (hit under multiple misses), HW prefetch	Out-of-order core can overlap L1-miss/L2-hit.
Parallelism	2–4 cores; SMT optional (with preferred-thread priority)	Games/AI/physics in threads, but single thread still primary.

Rejected: VLIW (irregular integer code, compatibility), coarse-grained MT (no gain for one thread), very wide issue (clock/power).

Scenario 4 — Scientific kernel engine (regular FP loops, huge arrays)

Constraints: floating-point throughput, long-latency FP ops (add 3, mult 6, divide 24/25), matrices much larger than cache, predictable loops.

Block	Choice	Why
FP units	Fully pipelined adder (4-stage) and multiplier (7-stage); **pipelined or duplicated divider** if divides are frequent	Initiation interval 1 for add/mult; divider with interval 25 would serialise divides.
Scheduling	Compiler loop unrolling + software pipelining on a VLIW/static design, *or* Tomasulo with renaming	Independent iterations, known latencies: unrolling reaches 3.5 cycles/iter vs 9. Tomasulo is better if latencies vary (cache misses).
Branches	BTFNT or 2-bit BHT; delay slot from target	Loop back-edges almost always taken.
Cache software	Blocking, interchange, fusion, array merging	Capacity/conflict misses: interchange cut misses 93.75%; blocking reduces capacity misses to $2N^3/B$.
Cache hardware	Stream-buffer prefetch, non-blocking cache, 64 B blocks	Sequential/stride access, plentiful bandwidth.
Memory	Wide or heavily interleaved banks	Bandwidth-bound; bank count ≥ access cycles.
Parallelism	Multiple cores (DLP across data), SIMD/vector†	Same operation on lots of data.

Rejected: large BTB/predictor budget (loops are easy), heavy speculation machinery (little control-flow irregularity), SMT (FP code showed least SMT gain and most cache conflicts).

Scenario 5 — Low-cost embedded controller

Constraints: price is the emphasis, fixed firmware, modest performance, limited power.

Design: classic 5-stage in-order pipeline with forwarding and **early branch resolution in ID** (1-cycle penalty); **static** prediction (predict-not-taken, or BTFNT/profile-driven) plus **one delay slot** filled by the compiler (about 50% useful); small direct-mapped I- and D-caches (small and simple ⇒ fast hit, low area), maybe a single level; software techniques (loop unrolling, loop interchange, blocking) since the firmware is compiled offline; no SMT, no multiple issue. If a compiler team and fixed firmware exist and the application is a regular numeric kernel, **VLIW** reduces control hardware, but only because *binary compatibility is irrelevant.*
Why: price emphasis ⇒ minimal hardware; static techniques need no predictor memory; software cache optimisations cost zero hardware. **Rejected:** Tomasulo+ROB, BTB, trace cache, victim cache (area).

Scenario 8 — Numeric cache comparison (practice, constructed numbers)

Problem. L1 miss penalty 40 cc, hit time 1 cc. Option A: 16 KB direct-mapped, miss rate 3.0%. Option B: 16 KB 2-way, miss rate 2.6%, clock stretched by 10% (slide: 2-way ×1.10). Option C: option A plus a 4-entry victim cache removing 40% of misses (slide range 20–95%). Which is best?

Solution (AMAT in direct-mapped cycles). A: $1 + 0.030 \times 40 = 2.20$. B: $1.10 + 0.026 \times 40 = 2.14$ — only 2.7% better *in AMAT*, but the 10% longer clock also slows *every non-memory instruction*, so B is a net loss. C: $1 + 0.030 \times 0.6 \times 40 = 1.72$ (assuming the victim probe hides inside the normal miss path, per the slide). **Choose C**: keeps the direct-mapped hit time (clock) and removes conflict misses cheaply. Quote the slide table: at 16 KB, 1-way has the lowest AMAT.

Scenario 9 — Numeric branch-handling comparison (practice, constructed numbers)

Problem. Deep pipeline (R4000-like): branch penalty 3 cycles, 20% of instructions are branches, 60% of branches are taken, base CPI 1.0. Compare (a) stall always, (b) predict-not-taken, (c) 2-bit BHT+BTB 90% accurate, (d) correlating/two-level predictor 95% accurate.

Solution.

Scheme	Extra CPI	CPI
(a) Stall always	$0.20 \times 3 = 0.60$	1.60
(b) Predict not taken (wrong when taken)	$0.20 \times 0.60 \times 3 = 0.36$	1.36
(c) 2-bit BHT + BTB, 90%	$0.20 \times 0.10 \times 3 = 0.06$	1.06
(d) Two-level, 95%	$0.20 \times 0.05 \times 3 = 0.03$	1.03

Speedup of (c) over (a): $1.60/1.06 \approx 1.5$; (d) over (c): ≈ 1.03. **Conclusion:** in a deep pipe the dynamic predictor is mandatory; the extra complexity of (d) is justified only if area allows (diminishing returns), or if branch frequency is higher or the pipeline deeper.

7.4 Conflict resolution: when two constraints disagree

Conflict	Resolution
High clock vs. low CPI	Deeper pipeline raises clock but adds load/branch delay; compensate with prediction and forwarding. Decide by the *product* IC×CPI×T_{clk}.
Low miss rate vs. fast hit	Large/associative cache cuts misses but stretches the clock; use small DM L1 + big associative L2; victim cache/way prediction as compromises.
Throughput vs. single-thread latency	Fine-grained MT/SMT raise throughput but slow a thread; add priority or fewer threads; or heterogeneous cores.
Performance vs. power	ILP/speculation/prefetch cost energy; prefer TLP/multicore and DVFS; energy per task is the right metric.
Compiler simplicity vs. hardware simplicity	Dynamic scheduling = simple compiler, complex hardware; VLIW = opposite. Choose by compatibility and cost.
Bandwidth vs. latency	Buy bandwidth (wide/interleaved memory, banks); hide latency (OoO, non-blocking, MT, prefetch).
Compatibility vs. simplicity	Compatibility ⇒ superscalar; fixed product ⇒ VLIW.

7.5 Final ten-minute exam checklist

1. Did I state the machine class and its priority (energy / cost / throughput / availability / single-thread)?
2. For *every* block (core, predictor, L1, L2/L3, memory, threads) did I give a choice *and* a reason tied to a constraint?
3. Did I give at least one number (AMAT, CPI, stalls/instr, speedup)?
4. Did I check feature dependencies (Section above)? (OoO ⇒ non-blocking; speculation ⇒ ROB+predictor; SMT ⇒ OoO.)
5. Did I state costs (area, power, clock, complexity) of each choice?
6. Did I list rejected alternatives and why?
7. If the scenario hints at compatibility, did I avoid VLIW (or justify)?

A Formula and Number Sheet

Numbers worth quoting that are not repeated in the sections above. † = standard formula not spelled out in the slide text.

A.1 Performance, power, cost, dependability

Item	Formula
CPU time	IC × CPI × T_{clk}
Average CPI	$\sum_i (\text{IC}_i \times \text{CPI}_i)/\text{IC}$
Speedup	$\text{Time}_{old}/\text{Time}_{new}$
Amdahl†	$1/[(1-f) + f/s]$
Dynamic energy	$\frac{1}{2}CV^2$
Dynamic power	$\frac{1}{2}CV^2 f$
Static power	$I_{static} \times V$ (reduce by power gating)
Die yield†	wafer yield × (1 + defects/area × die area)$^{-N}$, N = 11.5–15.5
MTBF; availability	MTBF = MTTF + MTTR; Availability = MTTF/MTBF
Price vs. volume	−10% per doubling of volume

A.2 Memory

Item	Formula / value
AMAT	$t_{hit} + m \times P$
Two-level AMAT	$t_{L1} + m_{L1}(t_{L2} + m_{L2}^{loc} P_{L2})$; global $m_{L2} = m_{L1} m_{L2}^{loc}$
Stalls/instruction (two-level)	$\text{misses}_{L1}/\text{instr} \times t_{L2} + \text{misses}_{L2}/\text{instr} \times P_{L2}$
Slide example	$1 + 0.04(10 + 0.5 \times 100) = 3.4$; stalls $(3.4 - 1) \times 1.5 = 3.6$
L2 associativity example	DM $10 + 0.25 \times 100 = 35$; 2-way $10 + 0.20 \times 100 = 30$
CPI with memory	$\text{CPI}_{base} + \text{refs/instr} \times m \times P$ (e.g. $2 + 1.2 \times 0.03 \times 64 = 4.3$)
Block-size penalty model	80 cc + 2 cc per 16 B: 82/84/88/96/112 for 16/32/64/128/256 B
Best block per cache (AMAT)	4K: 32 B; 16K, 64K, 256K: 64 B
Associativity clock factors	×1.10 (2-way), ×1.12 (4-way), ×1.14 (8-way)
2:1 rule	2-way of size $N/2 \approx$ DM of size N (caches < 128 KB)
Victim cache	20–95% miss reduction, 4 KB DM, 1–5 entries
Way prediction accuracy	$> 85\%$ (Alpha 21264, 2-way I-cache)
Prefetch AMAT	$t + m\,p\,t_p + m(1-p)P$; example 2.41525
Stream buffers	ISB 1/4/16 blocks catch 15–25%/50%/72% of I-misses
Loop interchange	1,048,576 → 65,536 (per array); −93.75%
Loop fusion	262,144 → 196,608 (−25%)
Blocking	$2N^3 + N^2 \to 2N^3/B + N^2$; 3 tiles of B^2 words must fit
Interleaving	bank = addr mod #banks; banks ≥ cycles to access a word
Memory-width CPI example	1-word 4.30; 2-word: simple 5.07 / interleaved 3.63 / wide 3.54; 4-word: 5.69 / 3.09 / 3.84
TLB	128–256 entries typical; fully associative on high-end
Page size / VA split	4–8 KB; 32-bit VA = 20-bit VPN + 12-bit offset
DRAM refresh	∼8 ms, ∼1% of time

A.3 Threads and multicore

Item	Value
Pentium 4 SMT	1.01 (int rate), 1.07 (FP rate); pairs 0.90–1.58, avg 1.20
Power5 SMT	1.23 (int rate), 1.16 (FP rate); 0.89–1.41 for two copies
Power5 core size	+24% vs. Power4; physical regs 152→240; L2 1.44→1.92 MB
SMT threads per core	2, 4, or sometimes 8 (Power5: 2)
Hardware thread switch	vs. 100s–1000s of clocks for a process switch
General SMT gain	up to 30%

B Source Map, Gaps and Glossary

B.1 Which deck covers what

ca01: fundamentals, power, cost. **ca03_noor**: pipelining, FP loop, unrolling, dependences, scoreboard, Tomasulo intro, VLIW vs. superscalar. **ca03_noorNew**: FP pipeline latencies, R4000. **ca04**: branch prediction, speculation/ROB, multiple issue, limits. **ca05**: all cache techniques, main memory, virtual memory. **ca06**: limits to ILP, multithreading, SMT, Power5. **ca08**: multicore, SMT vs. multicore. **lec06-dynasched**: Tomasulo vs. scoreboard, speculative Tomasulo. **CachePerformance2**: way prediction, pseudo-associative, interchange/fusion/blocking numbers. **Cache-Misses**: 3 C's + coherence misses.

B.2 Gaps: topics in the course outline but *not* in the provided slides

If the exam touches these, supplement from the textbook (Hennessy & Patterson):

- Vector architectures, SIMD multimedia extensions, GPUs (only named as DLP).
- Centralised shared-memory vs. distributed shared-memory / directory-based coherence, snooping protocols (MSI/MESI), synchronisation, memory consistency models. (Only *coherence misses* are covered.)
- Warehouse-scale computers, request-level parallelism, cloud computing, physical infrastructure.

B.3 Known inconsistencies in the slides (avoid losing marks)

- **R4000 load delay:** one slide title says "x1", the table beneath it shows *two* stall cycles, and the older deck says "TWO cycle load latency". Use **2**.
- **Memory-width CPI example:** the slide labels buses "64-bit" and "128-bit" but the arithmetic treats them as 1-word and 2-word wide; quote the CPI values (4.30, 5.07, 3.63, 3.54, 5.69, 3.09, 3.84) rather than the labels.
- **Blocking numbers** (≈ 1.07 billion →≈ 2.1 million) come from a *simplified* model (one miss per column access, tile reuse); present them as illustrative.